

Modify RV32I Pipelined Processor and its Memory Management to execute Atomic Instructions

Samiksha Aveg, 510574673

Abstract—This paper outlines the improvements made to an open-source RISC-V (RV32I) 2-Stage Pipeline Processor, to incorporate two specific instructions: Load reservation (lr.w) and Store conditional (sc.w) from the "A"-Extension Instruction set. Alongside these additions, modifications were also made by adjusting the RAM memory module to support two write accesses and two read accesses concurrently, with memory address-based reservations for the precise execution of the two Atomic Series Instructions. The functionality and accuracy of this processor were then validated using 13 pre-existing System Verilog test files.

Index Terms—RV32I, "A"-Extension, Load Reservation, Store Conditional, RAM, memory-address based reservation

I. INTRODUCTION

THE RISC-V consists of several instruction sets such as - Register (R), Immediate (I), Branch (B), Jump (J) types. The NERV program being used already includes the aforementioned instructions in a 32-bit format and features a 2-stage pipeline. This serves as a foundation, which we have further enhanced by integrating two instructions from the "A"-extension set. Afterwards, RAM memory has been modified to support concurrent dual write accesses and dual read accesses. This has been placed under the NERV program within the same system verilog (.sv) file. The very key challenge of this experiment is synchronizing our updated 2-stage pipelined NERV program with the modified RAM memory. We will also be adding a reservation vector for each of the two cores, where the appropriate reservation (based on the address) is set for that core when the LR.W instruction is executed, while all reservations for that address are cleared when a SC.W instruction is set. Further, we will test the accuracy of our implementation by running it against several test files already provided with the research prompt and discuss key challenges.

II. BACKGROUND

A. RISC-V Instruction Set

The RISC-V (Reduced Instruction Set Computer – Five) architecture is an open-standard instruction set architecture (ISA) developed to enable clean-slate, modular, and extensible processor designs. Unlike proprietary ISAs, RISC-V is free to use and implement, encouraging innovation in academic, industrial, and open-source processor development.

RISC-V follows a load-store architecture with a fixed 32-bit instruction format for its base integer set, known as RV32I. The base instruction set can be augmented with

standard extensions to support additional capabilities such as integer multiplication and division (M), atomic operations (A). The RV32IA configuration combines the base 32-bit integer instructions (I) with the atomic extension (A) - which includes lr.w (load-reserved word) and sc.w (store-conditional word), provides the foundational primitives required for implementing locks, mutexes, and other concurrency constructs in multi-core environments.

The **Zalrsc** subset of the A-extension defines two atomic primitives [Appendix A]:

- 1) **lr.w rd, (rs1)** — Load the 32-bit word from address rs1 into rd and mark that address as reserved for that hart/core. This reservation will survive until a store conditional operation, otherwise it is cleared from the memory.
- 2) **sc.w rd, rs2, (rs1)** — It tries to store the word in rs2 to address rs1 only if the core still holds a valid reservation. If successful, memory is updated and store occurs and rd = 0; if failed, memory remains unchanged and rd = 1.
- 3) A reservation is invalidated if:
 - Another core executes a successful sc.w to the same address.
 - The core's reservation is explicitly cleared or replaced.
 - A memory coherence event or exception occurs.

B. Multi-core processors

In single-core systems, the processor executes instructions sequentially, meaning it can only execute one set of instructions at a time with all memory operations occurring in a single thread of control. While this may seem limiting, single-core processors can still perform well for basic tasks such as web browsing, word processing, and multimedia playback. With rise of modern applications such as, web servers, databases, needing concurrent memory access system and parallelism - it led to the invention of Multi-core processors.

A multi-core processor contains multiple processing units, or cores, on a single chip, allowing the processor to execute multiple sets of instructions simultaneously. Thus, greatly improving computational capabilities, overall performance, and multitasking capabilities. These are commonly found in modern computers and smartphones. But it needs a lot of work to implement proper synchronisation of instructions, so

that when multiple cores access and modify same memory locations it doesn't lead to data races, inconsistent views of memory, outputs and hard-to-reproduce bugs. Therefore, it is important that the foundation to implement Load-Reserved (lr.w) and Store-Conditional (sc.w) are safe and synchronised so that it can support higher-level primitives like mutual exclusion, semaphores and provide safe shared-memory communication.

III. ARCHITECTURE

A. Design Goals

Taking all the aforementioned factors into account, the objective is to enhance the current WSPR-SDR receiver design by addressing specific design goals, which include: After having foundational knowledge, following are the objectives we aim to achieve:

- A 2-stage pipelined RV32I processor incorporating Atomic instructions: LR.W and SC.W;
- Modify the RAM memory module to support two write accesses and two read accesses concurrently;
- Decide on a priority scheme when both attempt to write in the same cycle;
- Add a reservation vector for each core, for the LR.W and SC.W instructions.

B. Processor

The RISC-V NERV processor design is based on the foundations laid in previous labs (upto Lab-06). It features a 2-stage pipeline, consisting of Instruction Fetch and Decode as the first stage, followed by Execute and Memory Writeback in the second stage. This helps to efficiently process instructions from the R, I, B, and J extensions. Before modifying the NERV processor design to incorporate Atomic instructions, it is essential to understand the operation of this working pipeline. The pipeline sits between the General Purpose Registers (GPRs) and Immediate Select modules or between the Decode and Execute stages [Appendix B].

The processor computes the instructions in the given manner:

- 1) **Instruction Fetch and Decode (IF+ID):** It's main purpose is to fetch instructions from memory, decode its fields/operands and control signals for execution.
 - Step 1: Fetch instruction from instruction memory using the Program Counter (pc).
 - Step 2: Increment pc by 4 ($pc + 4$) to point to the next instruction.
 - Step 3: Instruction Register (IR) will decode its fields to determine its type and control signals. The extracted fields usually consists of opcode, rs1, rs2, rd and immediate fields
 - Step 4: Read values from the register operands (rs1, rs2) from the register file. These will be utilised for the upcoming ALU operations.

- Step 5: Immediate Generator extends and formats instruction bits as per the extension types (I, B, or J types).
- Step 6: Control Unit will decode the opcode and function bits to match with the opcode definitions in the file to set signals for ALU operations, memory access, jump, branch or register write-back.

At the end of this stage, all operands and control signals are ready and stored in the pipeline register between IF+ID and EX+MEM+WB.

2) Execute and Memory Writeback (EX+MEM+WB):

The purpose of this stage is to perform the necessary arithmetic or logical computation, memory operation, or branch/jump decision and write results back to the register file.

- Step 7: Arithmetic Logic Unit (ALU) performs required arithmetic, logical or comparison operations depending on the fetched instruction.
- Step 8: Adder and Logic will calculate the pc+imm for branch or jump targets, and evaluates branch conditions (BEQ, BNE, etc.)
- Step 9: Data Memory is used for load and store instructions to help in reading and writing of data to a specific computed memory address.
- Step 10: Multiplexer will help in selecting ALU Result based on specific instruction, such as memory read data for load instruction or return memory address for jump instructions as the value to be written to the destination register (rd).
- Step 11: The final result is stored into Destination Register File (rd) and it is returned if required.

After understanding the operation of the existing NERV module, several modifications were made to support for atomic instructions (LR.W and SC.W) as defined in the RISC-V "A" extension specifications [Appendix A].

First, the module interface was updated to include three new ports: `dmem_wr_is_cond` (control signal for store-conditional memory access), `dmem_resv` (control signal for load reservation when LR.W is executed), and `dmem_cond` (a status flag returning 1 for success or 0 for failure following an SC.W instruction).

Next, the opcode for the atomic memory operations was identified within the existing opcode definition section, defined as: `[OPCODE_AMO = 7'b01011111]`.

To detect and track atomic instructions within the pipeline, three internal control signals were introduced: `is_lr_q`, `is_sc_q`, and `is_sc_q_delayed`. These signals respectively indicate the execution of an LR.W instruction, an SC.W instruction, and a delayed pipeline stage for memory writeback after store-conditional in previous cycle. When performing an LR.W, `is_lr_q` is set to 1, and similarly `is_sc_q` is set to 1 when an SC.W is encountered. The delay signal `is_sc_q_delayed` is used to ensure that there is a delay before executing next instruction. This is to allow for proper memory writeback and update of register file in next cycle

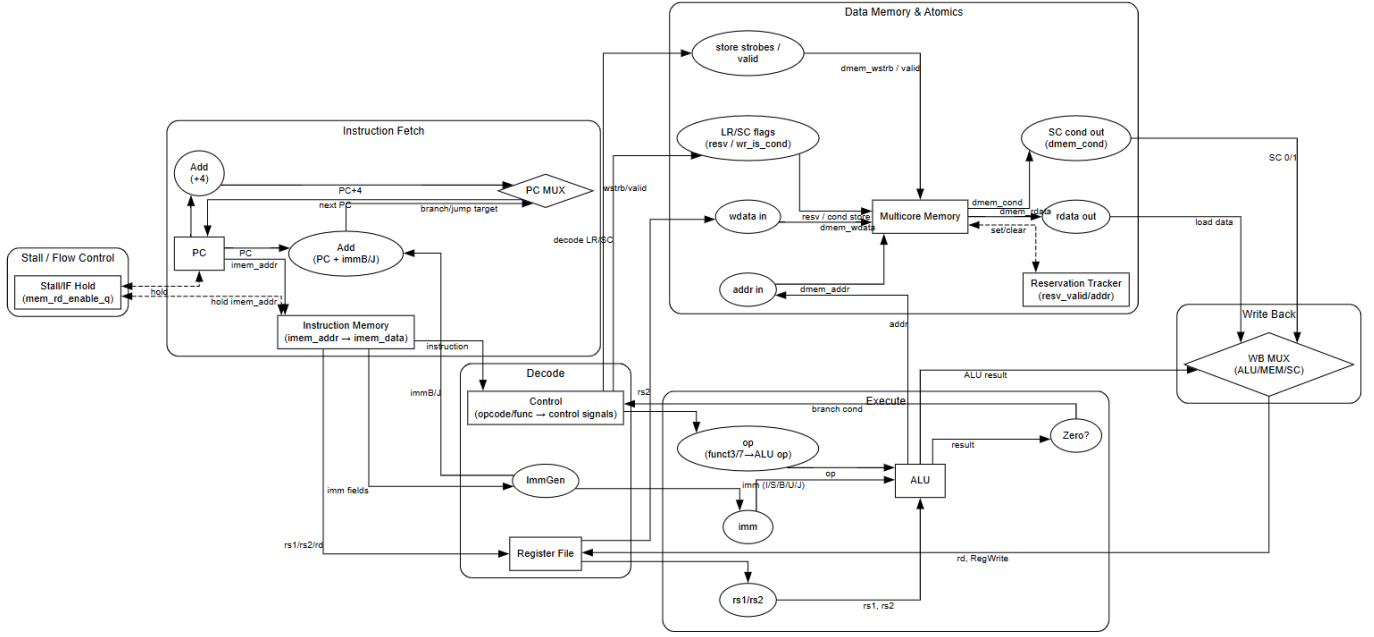


Fig. 1. 2-stage pipelined Atomic NERV Datapath with LR/SC Support (interacting with multicore_memory)

thus, ensuring correct timing and that correct values of that register are used in subsequent instructions.

After the main combinational logic, the signal `is_sc_q_delayed` was integrated into the pipeline stall cycle to maintain correct sequencing. The control outputs were then connected to the memory interface such that `dmem_resv` is assigned the value of `is_lr_q`, and `dmem_wr_is_cond` is assigned the value of `is_sc_q`.

Following the existing implementation of LOAD and STORE instructions, the similar was written for LR.W and SC.W. The combinational logic block selects the appropriate instruction based on various values of fields `{EX_insn[31:27], EX_insn_func3}`. To verify correct instruction decoding, the module was tested iteratively by temporarily disabling normal LOAD/STORE instructions and observing the atomic instruction behaviour.

Once the correct encoding combinations were identified, the functional behaviour was updated. The LR.W instruction now sets a reservation flag for the accessed address, while the SC.W instruction runs only if the reservation is valid. If successful, the value of destination register is updated 0; otherwise, it writes 1, indicating failure.

The newly added signals were also integrated into the memory access interface through `dmem_valid` signal, which asserts when any LOAD, STORE, LR.W, or SC.W instruction requires memory access. During the writeback stage, `dmem_cond` signal determines whether the store operation succeeds (writing to memory) or fails (skipping the write and updating the destination register accordingly).

The implementation was tested progressively after each modification using the provided `test-1` input file to ensure whether is operation is correct, appropriate error handling (especially no infinite looping), and cycle timing behaviour. Finally, all flags and control signals were reset to zero during

the processor reset phase to ensure consistent initialization. Lastly, the BRANCH and JUMP instructions haven't been removed from the program as it allows for better control over the management of cycle timings.

C. RAM Modifications and Reservation Vector Implementation

To support the updated RV32IA processor for multi-core atomic operations and parallel memory access, a *multicore_memory* module was developed. This design extends the original RAM interface [Appendix E] into a dual-hart/dual-core architecture to achieve our remaining objectives of modifying RAM, deciding priority scheme and adding reservation vector for each hart.

D. Dual-Hart RAM Architecture

The *multicore_memory* module consists of a shared memory array declared as a register block:

```
reg [7:0] mem [0:(1<MEM_ADDR_WIDTH)-1];
```

Two instruction address ports (`imem_addr0`, `imem_addr1`) and two data address ports (`dmem_addr0`, `dmem_addr1`) allow simultaneous memory access by Core0 and Core1, respectively. Each core can perform one read and one write operation per cycle, enabling parallel instruction fetch and data access.

In case of write conflicts where both cores attempt to modify memory block in same cycle, a **priority scheme** is applied where **Core0 is always given higher priority over Core1**, ensuring consistent operation and preventing race conditions.

E. Reservation Vector and Atomic Operation Tracking

Following the previous implementation of `lr.w` and `sc.w` semantics, a reservation vector for each core is added. Each core maintains a dedicated reservation entry consisting of a valid flag and an address identifier:

(resv_valid[0], resv_addr[0]) and (resv_valid[1], resv_addr[1]), for Core0 and Core1 respectively

When a `lr.w` instruction is executed, the corresponding `dmem_resv[i]` signal from the processor sets the reservation flag and stores the target address (indexed using `resv_id[i]`). This address represents the memory block on which the core has established a reservation.

During an `sc.w` instruction, the memory module verifies whether the stored reservation address matches the target memory address and that the reservation is still valid. If the conditions hold true, the store proceeds, and the signal `dmem_cond[i]` is asserted to 1, indicating a successful conditional write. Otherwise, the store is aborted, and `dmem_cond[i]` is returned as 0 to the core. After either case, all reservations corresponding to that memory block are invalidated for both cores to prevent further inconsistent writes.

Additionally, when any core performs a normal store (non-conditional), all active reservations for the same memory block are cleared, ensuring mutual exclusion and coherence across the cores. Further, a schematic diagram has been drawn to enable better understanding of the processor's latest working datapath/flowpath. [Refer to Fig. 1]

F. Dataflow Path

We have tried our best to follow the dataflow timeline [Appendix C] between the processor cores is as follows:

- **Instruction Fetch Path:** Both cores independently read instruction words from memory using their respective `imem_addr` signals. These reads are combinational with Core0 always given preference, thus no racing conditions.
- **Data Access Path:** Each core issues data access requests via `dmem_valid`, providing the address, write strobes, and data. If `dmem_resv[i]` is high, the memory treats the request as a load-reserve. If `dmem_wr_is_cond[i]` is high, the access is treated as a store-conditional.
- **Reservation Handling:** Upon `lr.w`, the memory captures the reservation address in `resv_addr[i]`. On `sc.w`, it validates the reservation and either commits the write or signals failure through `dmem_cond[i]`.

This structure ensures that atomic memory regions are correctly synchronized between cores and that no two cores can perform conflicting writes on the same reservation block concurrently.

IV. RESULTS

The latest version of the program has been validated using several input test files so as to better understand its overall functionality and to identify potential improvements needed

so that it can return the expected outcomes. Even after extensive refinements and rearrangement of the code, at the time of writing this paper - the program still had persistent synchronisation issues between the RV32IA (nerv module) and the RAM (multicore_memory) [Appendix C]. This issue mainly relates to the pipeline timing coordination due to which SC.W doesn't write the updated values back in the register file, but rather writes the outdated one. Thus, failing to display correct output after concurrent races/interleave scenarios which was present in majority of the testfiles. While testing our program, there were few discrepancies noticed such as an error was rectified in Makefile where 'test11-hart2-inside-hart1.hex' should have been 'test11-hart1-interleave-hart2.hex' in order to actually test our program the said file, our program showed no defined behaviours for same cycle read and write were when running against test4 and test5 files. The performance of this program across various tests can be seen under Appendix section where each test is highlighted alongwith its GTKwave simulation output and result logs [Appendix D][Appendix F][Appendix G]

Moreover, to avoid potential pipeline hazards, bypassing of instructions has been implemented where if certain instructions like LOAD, STORE, BRANCH, JUMP are executed then the program counter isn't updated for the subsequent cycle so that memory updation can happen. Further, hazards (like race conditions or breaking of pipeline) for the atomic memory operations and shared memory are managed by implementing a reservation mechanism which prioritises core0 over core1 when accessing shared memory in the same cycle. Additionally, if execution of an SC.W fails due to invalid reservation, the core gets the results immediately through signal - `dmem_cond[0]` (for core0)/`dmem_cond[1]` (for core1) and continues performing rest of the instructions without flushing of pipeline. The RAM module uses a synchronous `always_ff` block to update reservations every clock cycle and when a global reset occurs, all reservation flags are cleared, returning the system to a clean state.

V. DISCUSSION AND CONCLUSION

This experiment involving the implementation of atomic instructions and the modification of RAM to support a reservation vector for each core in shared memory environment has been quite fruitful. The atomic instructions function correctly for most cases, able to update and write to the register files. However, synchronisation between the RV32IA architecture and the shared multicore memory system proved to be very difficult with most of the time trying to ensure that no garbage value or values from previous cycle are being read by other registers in the current cycle. Additional development and testing is required to fully resolve this persistent synchronisation issue of instruction timing and clock cycle alignment of the two modules. Future improvements may include extending the current processor design to include fairness between the prioritisation of the two cores, adding additional pipeline stages and work enhancing its overall performance and structure .

APPENDIX A
ATOMIC INSTRUCTION FORMAT (RISC-V)

	0		6	7		11	12		14	15		19	20		24	25	26	27		31	
LR.W/D { SC.W/D {	funct5		aq	rl	rs2			rs1			funct3			rd			opcode				
			ordering		0			addr			width			dest			AMO				
			ordering		src			addr			width			dest			AMO				

Fig. 2. RISC-V A-extension encodings for LR.W/D and SC.W/D.

APPENDIX B

SCHEMATIC DIAGRAM OF PIPELINED BASE RV32 PROCESSOR

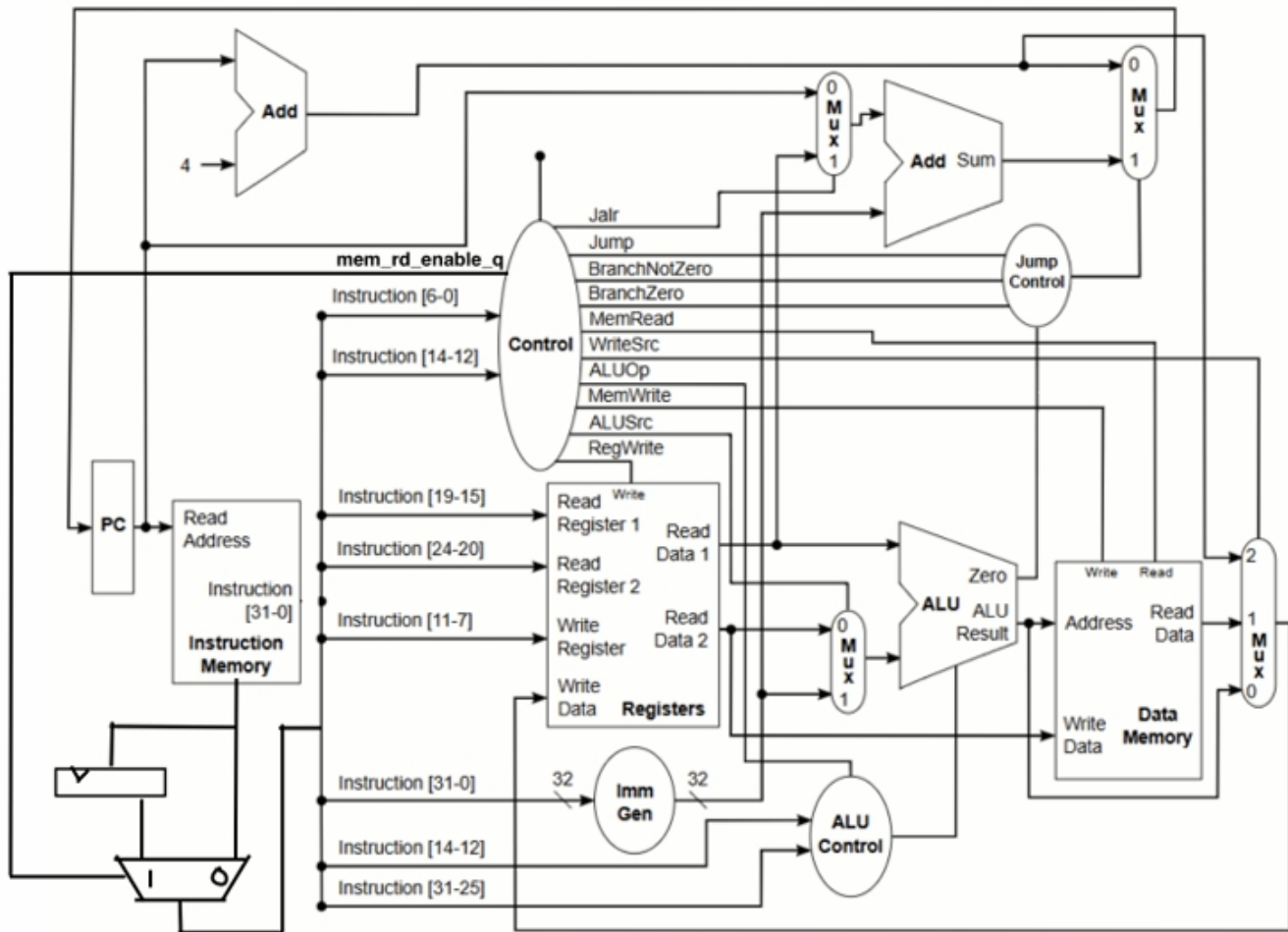


Fig. 3. Schematic of the RV32 NERV module (2-stage pipeline).

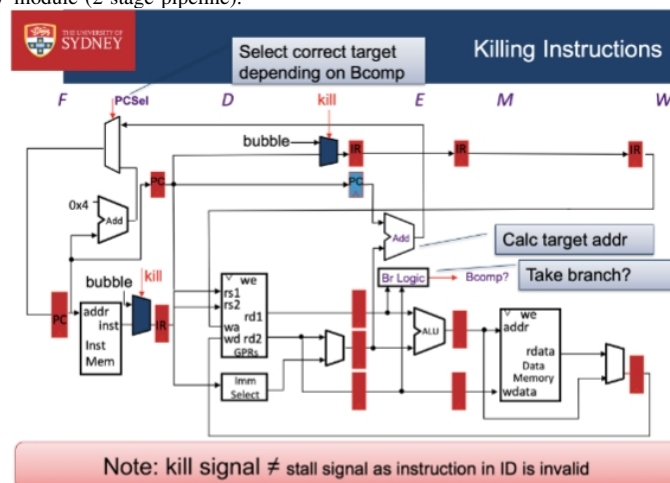


Fig. 4. Schematic of stall in RV32 NERV module (2-stage pipeline).

APPENDIX C

DATAFLOW TIMELINE OF LR.W AND SC.W IN SHARED MEMORY

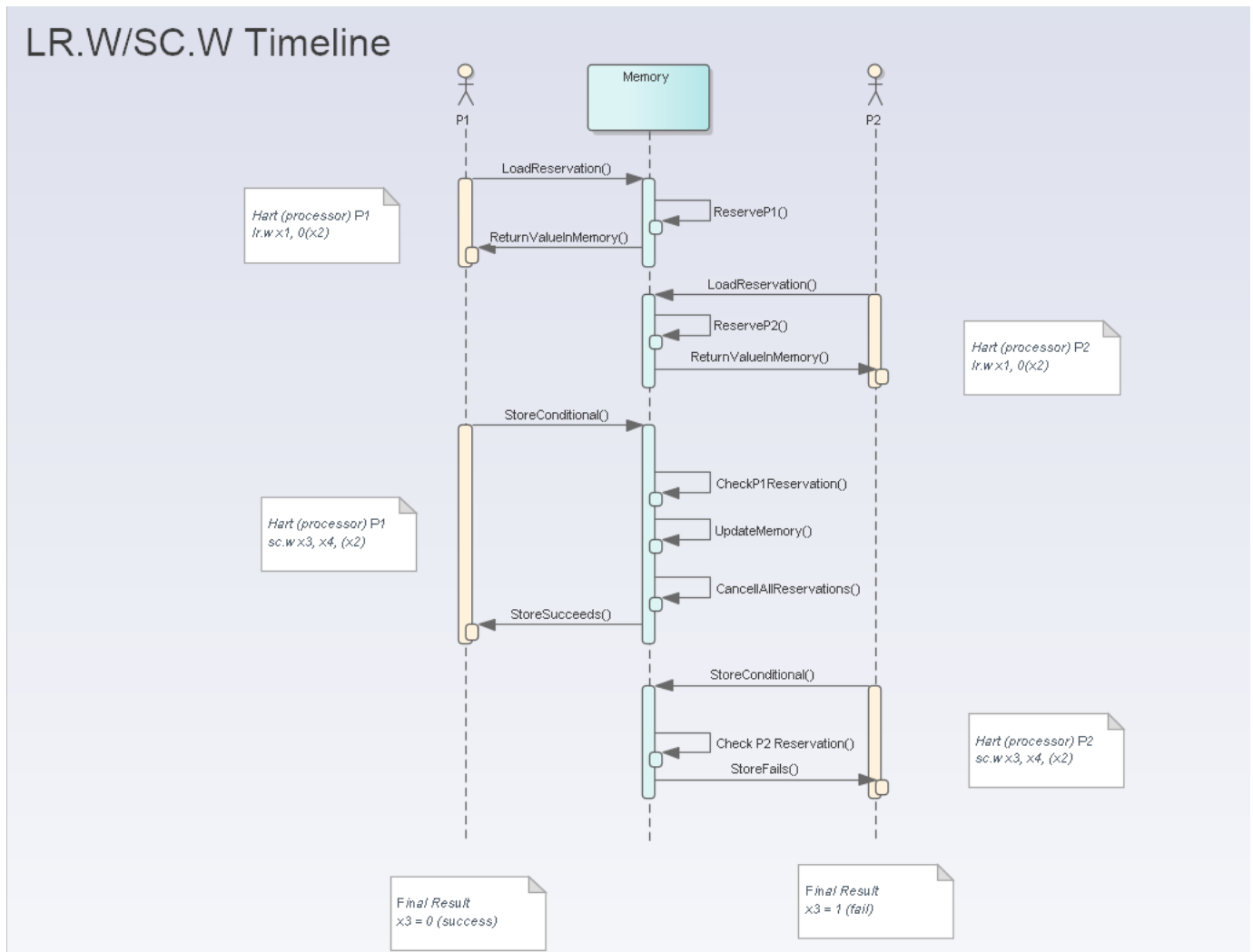


Fig. 5. Dataflow timeline for LR.W/SC.W in a shared-memory system.

APPENDIX D

SIMULATION RESULTS FOR NERV-ATOMIC TESTS

```

Source file: test1-hart1-only.s      Sim file:
test1-hart1-only.log
h0:x6 expected 192 got 1258291603
Total failures = 1 of 3 tests

Source file: test2-hart2-only.s      Sim file:
test2-hart2-only.log
h1:x6 expected 193 got 19
Total failures = 1 of 3 tests

Source file: test3-samecode-bothharts.s      Sim
file: test3-samecode-bothharts.log
h0:x6 expected 192 got 201326867
h1:x5 expected 1 got 0
h1:x6 expected 192 got 201326867
Total failures = 3 of 8 tests

Source file: test4-race-hart1-slowslow.s      Sim
file: test4-race-hart1-slowslow.log
h0:x5 expected 0 got 1
h0:x6 expected 192 got 201326867
h1:x6 expected 193 got 201326867
Total failures = 3 of 7 tests

Source file: test5-race-hart2-slowslow.s      Sim
file: test5-race-hart2-slowslow.log
h0:x6 expected 192 got 134217875
h1:x5 expected 0 got 1
h1:x6 expected 193 got 134217875
Total failures = 3 of 9 tests

Source file: test6-race-hart1-slow.s      Sim file:
test6-race-hart1-slow.log
h0:x6 expected 193 got 201326867
h1:x6 expected 193 got 201326867
Total failures = 2 of 7 tests

Source file: test7-race-hart2-slow.s      Sim file:
test7-race-hart2-slow.log
h0:x6 expected 192 got 201326867
h1:x6 expected 192 got 201326867
Total failures = 2 of 7 tests

Source file: test8-samecode-2reserv.s      Sim file:
test8-samecode-2reserv.log
h0:x6 expected 192 got 201326867
h1:x6 expected 193 got 201326867
Total failures = 2 of 8 tests

Source file: test9-samecode-1reserv.s      Sim file:
test9-samecode-1reserv.log
h0:x6 expected 192 got 201326867
h1:x5 expected 1 got 0
h1:x6 expected 0 got 201326867
Total failures = 3 of 8 tests

Source file: test10-hart1-inside-hart2.s      Sim
file: test10-hart1-inside-hart2.log
h0:x6 expected 193 got 0
h1:x6 expected 193 got 201326867
Total failures = 2 of 7 tests

Source file: test12-hart1-interleave-hart2.s      Sim
file: test12-hart1-interleave-hart2.log
h0:x6 expected 192 got 0
h1:x6 expected 192 got 0
Total failures = 2 of 7 tests

Source file: test13-hart2-interleave-hart1.s      Sim
file: test13-hart2-interleave-hart1.log
h0:x6 expected 193 got 0
h1:x6 expected 193 got 0
Total failures = 2 of 7 tests

```

APPENDIX E

ORIGINAL NERV-ATOMIC.SV

```

/*
 * NERV -- Naive Educational RISC-V Processor
 *
 * Copyright (C) 2020 Claire Xenia Wolf
 * <claire@yosyshq.com>
 *
 * Permission to use, copy, modify, and/or
 * distribute this software for any
 * purpose with or without fee is hereby granted,
 * provided that the above
 * copyright notice and this permission notice
 * appear in all copies.
 *
 * THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR
 * DISCLAIMS ALL WARRANTIES
 * WITH REGARD TO THIS SOFTWARE INCLUDING ALL
 * IMPLIED WARRANTIES OF
 * MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL
 * THE AUTHOR BE LIABLE FOR
 * ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL
 * DAMAGES OR ANY DAMAGES
 * WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR
 * PROFITS, WHETHER IN AN
 * ACTION OF CONTRACT, NEGLIGENCE OR OTHER
 * TORTIOUS ACTION, ARISING OUT OF
 * OR IN CONNECTION WITH THE USE OR PERFORMANCE OF
 * THIS SOFTWARE.
 */

module nerv #(
    parameter [31:0] RESET_ADDR = 32'h 0000_0000,
    parameter integer NUMREGS = 32
) (
    input clock,
    input reset,
    output trap,

    // we have 2 external memories
    // one is instruction memory
    output [31:0] imem_addr,
    input [31:0] imem_data,

    // the other is data memory
    output dmem_valid,
    output dmem_wr_is_cond,
    output [31:0] dmem_addr,
    output [ 3:0] dmem_wstrb,
    output [31:0] dmem_wdata,
    input [31:0] dmem_rdata,

    output dmem_resv, // reservations
    input dmem_cond // conditional
);

// registers, instruction reg, program counter,
// next pc
logic [31:0] regfile [0:NUMREGS-1];

// YOUR CODE HERE

endmodule

```

```

/*****/

module multicore_memory #(
    parameter MEM_ADDR_WIDTH = 17,
    parameter string fname,
    parameter RESV_BITS=12
) (
    input clock,
    input reset,

    input [31:0] imem_addr0,
    output reg [31:0] imem_data0,

    input dmem_valid0,
    input dmem_wr_is_cond0,
    input [31:0] dmem_addr0,
    input [ 3:0] dmem_wstrb0,
    input [31:0] dmem_wdata0,
    output [31:0] dmem_rdata0,

    input [31:0] imem_addr1,
    output reg [31:0] imem_data1,

    input dmem_valid1,
    input dmem_wr_is_cond1,
    input [31:0] dmem_addr1,
    input [ 3:0] dmem_wstrb1,
    input [31:0] dmem_wdata1,
    output [31:0] dmem_rdata1,

    input [1:0] dmem_resv, // reservations
    output [1:0] dmem_cond // conditional
);

// THE ACTUAL MEMORY
reg [7:0] mem [0:(1<<MEM_ADDR_WIDTH)-1];

// YOUR CODE HERE

endmodule

```

APPENDIX F

ORIGINAL MAKEFILE

```
# NERV -- Naive Educational RISC-V Processor
#
# Copyright (C) 2020 N. Engelhardt
# <nak@yosyshq.com>
# Copyright (C) 2020 Claire Xenia Wolf
# <claire@yosyshq.com>
#
# Permission to use, copy, modify, and/or
# distribute this software for any
# purpose with or without fee is hereby granted,
# provided that the above
# copyright notice and this permission notice
# appear in all copies.
#
# THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR
# DISCLAIMS ALL WARRANTIES
# WITH REGARD TO THIS SOFTWARE INCLUDING ALL
# IMPLIED WARRANTIES OF
# MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL
# THE AUTHOR BE LIABLE FOR
# ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL
# DAMAGES OR ANY DAMAGES
# WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR
# PROFITS, WHETHER IN AN
# ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS
# ACTION, ARISING OUT OF
# OR IN CONNECTION WITH THE USE OR PERFORMANCE OF
# THIS SOFTWARE.

TOOLCHAIN_PREFIX?=riscv64-unknown-elf-
VERILATOR = verilator

# MacOS/Windows w/ docker desktop
#RDOCKER = docker run --platform linux/amd64 -it -e
# DISPLAY=host.docker.internal:0 -v `pwd`:/config
# phwl/elec3608-cad:latest

# Linux w/o SNAP & docker desktop
RDOCKER = docker run --platform linux/amd64 -it
--net host -e DISPLAY=$(DISPLAY) -v
`pwd`:/config phwl/elec3608-cad:latest

# this can be simply overridden using make
FIRMWAREFILE=... all
FIRMWAREFILE=test1-hart1-only.hex

# logging file name
LOGFILE='basename $(FIRMWAREFILE) .hex'.log
ASMFILE='basename $(FIRMWAREFILE) .hex'.s
RESULTFILE=results.log

# all tests
ALLFIRMWARE=test1-hart1-only.hex
test2-hart2-only.hex
test3-samecode-bothharts.hex \
test4-race-hart1-slowslow.hex
test5-race-hart2-slowslow.hex \
test6-race-hart1-slow.hex
test7-race-hart2-slow.hex
test8-samecode-2reserv.hex \
test9-samecode-1reserv.hex \
test10-hart1-inside-hart2.hex
test11-hart1-interleave-hart2.hex \
test12-hart1-interleave-hart2.hex
test13-hart2-interleave-hart1.hex
test10-hart1-inside-hart2.hex
test11-hart2-inside-hart1.hex \
test12-hart1-interleave-hart2.hex
test13-hart2-interleave-hart1.hex

all: $(FIRMWAREFILE) test_verilator

allresults:
```

```
$(foreach ff, $(ALLFIRMWARE), make result
FIRMWAREFILE=$(ff);)
@echo
"-----"
@echo "Results..."
@cat $(RESULTFILE)

result : all
@python checkregs.py $(LOGFILE) $(ASMFILE)
| tee --append $(RESULTFILE)

test_verilator: testbench_verilator
@./obj_dir/Vtestbench +vcd > $(LOGFILE)

testbench_verilator: $(FIRMWAREFILE)
testbench-2hart.sv nerv-atomic.sv testbench.cpp
@$(VERILATOR) --cc --exe -Wno-lint -trace
--top-module testbench \
-Gfirmwarefile="$(FIRMWAREFILE)" \
testbench-2hart.sv nerv-atomic.sv
testbench.cpp
@$(MAKE) -C obj_dir -f Vtestbench.mk

%.elf: %.s
@$(TOOLCHAIN_PREFIX)gcc -march=rv32ia
-mabi=ilp32 -Os -Wall -Wextra
-Wl,-Bstatic,-T,sections.lds,--strip-debug
-ffreestanding -nostdlib -o $$@ $$^

%.hex: %.elf
@$(TOOLCHAIN_PREFIX)objcopy -O verilog $< $@

show:
@gtkwave testbench.vcd testbench.gtkw >>
gtkwave.log 2>&1

rundocker:
$(RDOCKER)

clean:
rm -rf firmware.elf firmware.hex testbench
testbench.vcd gtwave.log
rm -rf disasm.o disasm.s checks/ cexdata/
obj_dir
rm -rf *.log *.asc *.json *.hex
```

APPENDIX G

SIMULATION OUTPUT OF TESTFILES

Label: test1-hart1-only.s

```

/*
 * NERV -- Naive Educational RISC-V Processor
 *
 * Copyright (C) 2020 Claire Xenia Wolf
 * <claire@yosyshq.com>
 *
 * Permission to use, copy, modify, and/or
 * distribute this software for any
 * purpose with or without fee is hereby granted,
 * provided that the above
 * copyright notice and this permission notice
 * appear in all copies.
 *
 * THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR
 * DISCLAIMS ALL WARRANTIES
 * WITH REGARD TO THIS SOFTWARE INCLUDING ALL
 * IMPLIED WARRANTIES OF
 * MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL
 * THE AUTHOR BE LIABLE FOR
 * ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL
 * DAMAGES OR ANY DAMAGES
 * WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR
 * PROFITS, WHETHER IN AN
 * ACTION OF CONTRACT, NEGLIGENCE OR OTHER
 * TORTIOUS ACTION, ARISING OUT OF
 * OR IN CONNECTION WITH THE USE OR PERFORMANCE OF
 * THIS SOFTWARE.
 */

/**
 * @brief Test a single LR/SC instruction on hart0
 * @author Rich Rademacher
 */

.section .text_H0,"ax",@progbits /* this H0
    section is "allocatiable, program code,
    executable" */
.global main
.global _start
_start:
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
    should always change after SC.W */
    addi x4, zero, 0xc0 /* new value to put in
    memory, 'core0' */
    lr.w x0, (x3)
    sc.w x5, x4, (x3)
    /* sc.w rs2, rs1, rd */
    lw x6, (x3) /* read what is actually
    in memory */
    ebreak

/* expected result:
    expect h0:x3 = 1200
    expect h0:x4 = 0xc0
    expect h0:x6 = 0xc0
 */

/**
 * @brief Do nothing on Hart1
 */
.section .text_H1,"ax",@progbits /* this H1
    section is "allocatiable, program code,
    executable" */
.global _start1
_start1:
    nop
    nop
    ebreak

```

Label: test1-hart1-only logfile

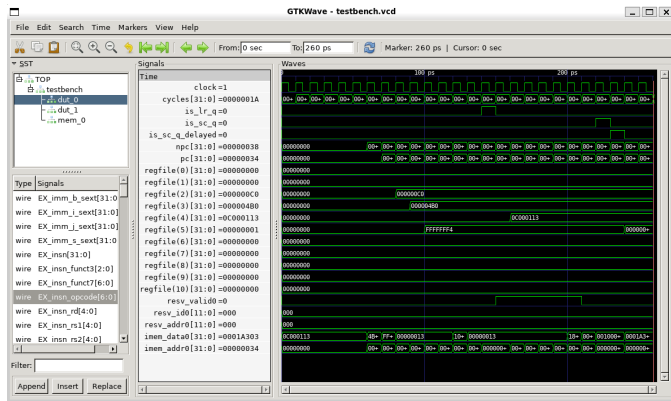
```

Built with Verilator 4.038 2020-07-11.
Recommended: Verilator 4.0 or later.
regfile[10]= 0
regfile[10]= 0
regfile[10]= 0
regfile[10]= 0
regfile[10]= 0
regfile[10]= 0
regfile[10]= 0
Simulated 17 cycles
h0:x0 =0x0
h0:x1 =0x0
h0:x2 =0x0
h0:x3 =0x4b0
h0:x4 =0xc0
h0:x5 =0x0
h0:x6 =0x4b000193
h0:x7 =0x0
h0:x8 =0x0
h0:x9 =0x0
h0:x10=0x0
h0:x11=0x0
h0:x12=0x0
h0:x13=0x0
h0:x14=0x0
h0:x15=0x0
h0:x16=0x0
h0:x17=0x0
h0:x18=0x0
h0:x19=0x0
h0:x20=0x0
h0:x21=0x0
h0:x22=0x0
h0:x23=0x0
h0:x24=0x0
h0:x25=0x0
h0:x26=0x0
h0:x27=0x0
h0:x28=0x0
h0:x29=0x0
h0:x30=0x0
h0:x31=0x0
h1:x0 =0x0
h1:x1 =0x0
h1:x2 =0x0
h1:x3 =0x0
h1:x4 =0x0
h1:x5 =0x0
h1:x6 =0x0
h1:x7 =0x0
h1:x8 =0x0
h1:x9 =0x0
h1:x10=0x0
h1:x11=0x0
h1:x12=0x0
h1:x13=0x0
h1:x14=0x0
h1:x15=0x0
h1:x16=0x0
h1:x17=0x0
h1:x18=0x0
h1:x19=0x0
h1:x20=0x0
h1:x21=0x0
h1:x22=0x0
h1:x23=0x0
h1:x24=0x0
h1:x25=0x0
h1:x26=0x0
h1:x27=0x0
h1:x28=0x0
h1:x29=0x0
h1:x30=0x0
h1:x31=0x0
- testbench-2hart.sv:164: Verilog $finish

```

```
regfile[10]=      0
regfile[10]=      0
```

Label: test1-hart1-only gtkwave



```
.section .text_H1,"ax",@progbits /* this H1
section is "allocatiable, program code,
executable" */
.global _start1
_start1:
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
should always change after SC.W */
    addi x4, zero, 0xc1 /* new value to put in
memory, 'core1' */
    lr.w x0, (x3)
    sc.w x5, x4, (x3)
    lw x6, (x3) /* read what is actually
in memory */
    ebreak

/* expected result:
expect h1:x3 = 1200
expect h1:x4 = 0xc1
expect h1:x6 = 0xc1
*/
```

Label: test2-hart2-only logfile

Label: test2-hart2-only.s

```
/*
 * NERV -- Naive Educational RISC-V Processor
 *
 * Copyright (C) 2020 Claire Xenia Wolf
 * <claire@yosyshq.com>
 *
 * Permission to use, copy, modify, and/or
 * distribute this software for any
 * purpose with or without fee is hereby granted,
 * provided that the above
 * copyright notice and this permission notice
 * appear in all copies.
 *
 * THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR
 * DISCLAIMS ALL WARRANTIES
 * WITH REGARD TO THIS SOFTWARE INCLUDING ALL
 * IMPLIED WARRANTIES OF
 * MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL
 * THE AUTHOR BE LIABLE FOR
 * ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL
 * DAMAGES OR ANY DAMAGES
 * WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR
 * PROFITS, WHETHER IN AN
 * ACTION OF CONTRACT, NEGLIGENCE OR OTHER
 * TORTIOUS ACTION, ARISING OUT OF
 * OR IN CONNECTION WITH THE USE OR PERFORMANCE OF
 * THIS SOFTWARE.
 */

/**
 * @brief Hart0: Do nothing on this hart
 * @author Rich Rademacher
 */

.section .text_H0,"ax",@progbits /* this H0
section is "allocatiable, program code,
executable" */
.global main
.global _start
_start:
    nop
    nop
    ebreak

/**
 * @brief Hart1 attempts to get a single LR/SC
 * variable from memory
 */
```

Built with Verilator 4.038 2020-07-11.

Recommended: Verilator 4.0 or later.

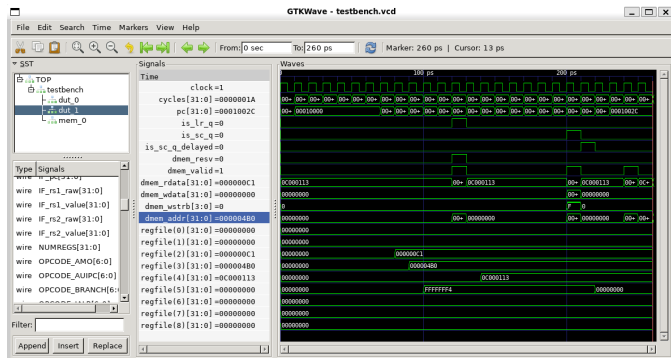
```
regfile[10]=      0
regfile[10]=      0
regfile[10]=      0
regfile[10]=      0
regfile[10]=      0
regfile[10]=      0
regfile[10]=      0
regfile[10]=      0
Simulated 17 cycles
h0:x0 =0x0
h0:x1 =0x0
h0:x2 =0x0
h0:x3 =0x0
h0:x4 =0x0
h0:x5 =0x0
h0:x6 =0x0
h0:x7 =0x0
h0:x8 =0x0
h0:x9 =0x0
h0:x10=0x0
h0:x11=0x0
h0:x12=0x0
h0:x13=0x0
h0:x14=0x0
h0:x15=0x0
h0:x16=0x0
h0:x17=0x0
h0:x18=0x0
h0:x19=0x0
h0:x20=0x0
h0:x21=0x0
h0:x22=0x0
h0:x23=0x0
h0:x24=0x0
h0:x25=0x0
h0:x26=0x0
h0:x27=0x0
h0:x28=0x0
h0:x29=0x0
h0:x30=0x0
h0:x31=0x0
h1:x0 =0x0
h1:x1 =0x0
h1:x2 =0x0
h1:x3 =0x4b0
h1:x4 =0xc1
h1:x5 =0x0
h1:x6 =0x13
h1:x7 =0x0
h1:x8 =0x0
```

```

h1:x9 =0x0
h1:x10=0x0
h1:x11=0x0
h1:x12=0x0
h1:x13=0x0
h1:x14=0x0
h1:x15=0x0
h1:x16=0x0
h1:x17=0x0
h1:x18=0x0
h1:x19=0x0
h1:x20=0x0
h1:x21=0x0
h1:x22=0x0
h1:x23=0x0
h1:x24=0x0
h1:x25=0x0
h1:x26=0x0
h1:x27=0x0
h1:x28=0x0
h1:x29=0x0
h1:x30=0x0
h1:x31=0x0
- testbench-2hart.sv:164: Verilog $finish
regfile[10]=      0
regfile[10]=      0

```

Label: test2-hart2-only gtkwave



Label: test3-samecode-bothharts.s

```

/*
 * NERV -- Naive Educational RISC-V Processor
 *
 * Copyright (C) 2020 Claire Xenia Wolf
 * <claire@yosyshq.com>
 *
 * Permission to use, copy, modify, and/or
 * distribute this software for any
 * purpose with or without fee is hereby granted,
 * provided that the above
 * copyright notice and this permission notice
 * appear in all copies.
 */

/**
 * @brief test Two harts competing for the same
 *        reservation. Based on the priority
 *        you implement (H0=higher in this case) one
 *        hart should fail
 * @author Rich Rademacher
 */

.section .text_H0,"ax",@progbits /* this H0
    section is "allocatiable, program code,
    executable" */

```

```

.global main
.global _start
_start:
    addi x2, zero, 0xc0 /* new value to put in
        memory, 'core0' */
    addi x3, zero, 1200 /* memory address 0x1200
        */
    addi x5, zero, -12 /* dummy result...this
        should always change after SC.W */
    lr.w x4, (x3)
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is actually
        in memory */
    ebreak

/* expected result:
    expect h0:x2 = 0xc0 (core 0)
    expect h0:x3 = 1200
    expect h0:x5 = 0 (success)
    expect h0:x6 = 0xc0 (192)
*/

/**
 * @brief Hart1 code same as Hart0
 */
.section .text_H1,"ax",@progbits /* this H1
    section is "allocatiable, program code,
    executable" */
.global _start1
_start1:
    addi x2, zero, 0xc1 /* new value to put in
        memory, 'core1' */
    addi x3, zero, 1200 /* memory address
        0x1200 */
    addi x5, zero, -12 /* dummy result...this
        should always change after SC.W */
    lr.w x4, (x3)
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is
        actually in memory */
    ebreak

/* expected result:
    expect h1:x2 = 0xc1
    expect h1:x3 = 1200
    expect h1:x5 = 1 (fail)
    expect h1:x6 = 0xc0 (core 1 - failed sc.w )
*/

```

Label: test3-samecode-bothharts logfile

```

Built with Verilator 4.038 2020-07-11.
Recommended: Verilator 4.0 or later.
Simulated 17 cycles
h0:x0 =0x0
h0:x1 =0x0
h0:x2 =0xc0
h0:x3 =0x4b0
h0:x4 =0xc000113
h0:x5 =0x0
h0:x6 =0xc000113
h0:x7 =0x0
h0:x8 =0x0
h0:x9 =0x0
h0:x10=0x0
h0:x11=0x0
h0:x12=0x0
h0:x13=0x0
h0:x14=0x0
h0:x15=0x0
h0:x16=0x0
h0:x17=0x0
h0:x18=0x0
h0:x19=0x0
h0:x20=0x0

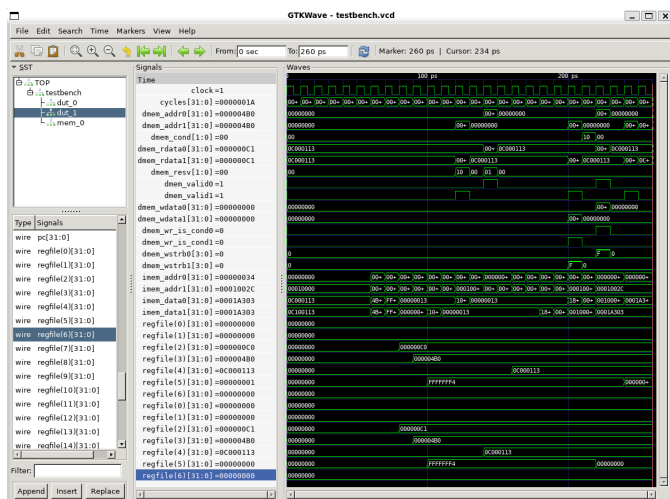
```

```

h0:x21=0x0
h0:x22=0x0
h0:x23=0x0
h0:x24=0x0
h0:x25=0x0
h0:x26=0x0
h0:x27=0x0
h0:x28=0x0
h0:x29=0x0
h0:x30=0x0
h0:x31=0x0
h1:x0 =0x0
h1:x1 =0x0
h1:x2 =0xc1
h1:x3 =0x4b0
h1:x4 =0xc000113
h1:x5 =0x0
h1:x6 =0xc000113
h1:x7 =0x0
h1:x8 =0x0
h1:x9 =0x0
h1:x10=0x0
h1:x11=0x0
h1:x12=0x0
h1:x13=0x0
h1:x14=0x0
h1:x15=0x0
h1:x16=0x0
h1:x17=0x0
h1:x18=0x0
h1:x19=0x0
h1:x20=0x0
h1:x21=0x0
h1:x22=0x0
h1:x23=0x0
h1:x24=0x0
h1:x25=0x0
h1:x26=0x0
h1:x27=0x0
h1:x28=0x0
h1:x29=0x0
h1:x30=0x0
h1:x31=0x0
- testbench-2hart.sv:164: Verilog $finish
regfile[10]= 0
regfile[10]= 0

```

Label: test3-samecode-bothharts gtkwave



Label: test4-race-hart1-slowslow

```

/*
 * NERV -- Naive Educational RISC-V Processor
 *
 * Copyright (C) 2020 Claire Xenia Wolf
 * <claire@yosyshq.com>
 *
 * Permission to use, copy, modify, and/or
 * distribute this software for any
 * purpose with or without fee is hereby granted,
 * provided that the above
 * copyright notice and this permission notice
 * appear in all copies.
 *
 */

/**
 * @brief Two harts competing for the same
 * reservation. Hart0 asks for LR/SC two cycles
 * later
 *
 * thus flushing the pipeline, and effectively
 * they don't compete at all.
 * @author Rich Rademacher
 */

.section .text_H0,"ax",@progbits /* this H0
section is "allocatable, program code,
executable" */
.global main
.global _start
_start:
    addi x2, zero, 0xc0 /* new value to put in
memory, 'core0' */
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
should always change after SC.W */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    lr.w x4, (x3)
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is actually in
memory */
    ebreak

/* expected result:
expect h0:x2 = 0xc0 (core 0)
expect h0:x3 = 1200
expect h0:x5 = 0 (success)
expect h0:x6 = 0xc0 (192)
*/

/**
 * @brief hart1 Tries immediately to get the memory
 * via LR/SC
 */

.section .text_H1,"ax",@progbits /* this H1
section is "allocatable, program code,
executable" */
.global _start1
_start1:
    addi x2, zero, 0xc1 /* new value to put in
memory, 'core1' */
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
should always change after SC.W */
    lr.w x4, (x3)
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is actually in
memory */
    ebreak

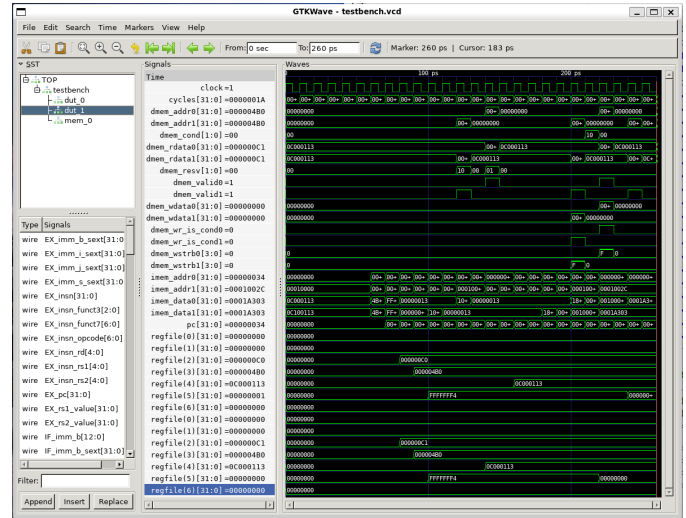
/* expected result:
expect h1:x2 = 0xc1 (core 1)
expect h1:x5 = 0 (success)
expect h1:x6 = 0xc1 (193)
*/

```

Label: test4-race-hart1-slowslow logfile

```
Built with Verilator 4.038 2020-07-11.
Recommended: Verilator 4.0 or later.
regfile[10]=      0
regfile[10]=      0
Simulated 19 cycles
h0:x0 =0x0
h0:x1 =0x0
h0:x2 =0xc0
h0:x3 =0x4b0
h0:x4 =0xc000113
h0:x5 =0x1
h0:x6 =0xc000113
h0:x7 =0x0
h0:x8 =0x0
h0:x9 =0x0
h0:x10=0x0
h0:x11=0x0
h0:x12=0x0
h0:x13=0x0
h0:x14=0x0
h0:x15=0x0
h0:x16=0x0
h0:x17=0x0
h0:x18=0x0
h0:x19=0x0
h0:x20=0x0
h0:x21=0x0
h0:x22=0x0
h0:x23=0x0
h0:x24=0x0
h0:x25=0x0
h0:x26=0x0
h0:x27=0x0
h0:x28=0x0
h0:x29=0x0
h0:x30=0x0
h0:x31=0x0
h1:x0 =0x0
h1:x1 =0x0
h1:x2 =0xc1
h1:x3 =0x4b0
h1:x4 =0xc000113
h1:x5 =0x0
h1:x6 =0xc000113
h1:x7 =0x0
h1:x8 =0x0
h1:x9 =0x0
h1:x10=0x0
h1:x11=0x0
h1:x12=0x0
h1:x13=0x0
h1:x14=0x0
h1:x15=0x0
h1:x16=0x0
h1:x17=0x0
h1:x18=0x0
h1:x19=0x0
h1:x20=0x0
h1:x21=0x0
h1:x22=0x0
h1:x23=0x0
h1:x24=0x0
h1:x25=0x0
h1:x26=0x0
h1:x27=0x0
h1:x28=0x0
h1:x29=0x0
h1:x30=0x0
h1:x31=0x0
- testbench-2hart.sv:164: Verilog $finish
regfile[10]=      0
regfile[10]=      0
```

Label: test4-race-hart1-slowslow gtkwave



Label: test5-race-hart2-slowslow.s

```
/*
 * NERV -- Naive Educational RISC-V Processor
 *
 * Copyright (C) 2020 Claire Xenia Wolf
 * <claire@yosyshq.com>
 *
 * Permission to use, copy, modify, and/or
 * distribute this software for any
 * purpose with or without fee is hereby granted,
 * provided that the above
 * copyright notice and this permission notice
 * appear in all copies.
 *
 */

/**
 * @brief Two harts competing for the same
 * reservation. Hart0 obtains the reservation and
 * writes back immediately. It is the
 * first
 * @author Rich Rademacher
 */

.section .text_H0,"ax",@progbits /* this H0
section is "allocatable, program code,
executable" */
.global main
.global _start
_start:
    addi x1, zero, 128
    addi x2, zero, 0xc0 /* new value to put in
memory, 'core0' */
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
should always change after SC.W */
    lr.w x4, (x3)
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is actually
in memory */
    ebreak

/* expected result:
expect h0:x1 = 128
expect h0:x2 = 0xc0 (core 0)
expect h0:x3 = 1200
expect h0:x5 = 0 (success)
expect h0:x6 = 0xc0 (193)
*/
```

```

/**
 * @brief Hart1 waits two cycles, thus the pipeline
 * is empty. Effectively Hart0 and Hart1 do not
 * compete at all
 */
.section .text_H1,"ax",@progbits /* this H1
section is "allocatable, program code,
executable" */
.global _start1
_start1:
    addi x1, zero, 128
    addi x2, zero, 0xc1 /* new value to put in
memory, 'core1' */
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
should always change after SC.W */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    lr.w x4, (x3)
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is actually in
memory */
    ebreak

/* expected result:
expect h1:x1 = 128
expect h1:x2 = 0xc1 (core 1 )
expect h1:x5 = 0 (success)
expect h1:x6 = 0xc1 (193)
*/

```

```

h1:x5 =0x1
h1:x6 =0x8000093
h1:x7 =0x0
h1:x8 =0x0
h1:x9 =0x0
h1:x10=0x0
h1:x11=0x0
h1:x12=0x0
h1:x13=0x0
h1:x14=0x0
h1:x15=0x0
h1:x16=0x0
h1:x17=0x0
h1:x18=0x0
h1:x19=0x0
h1:x20=0x0
h1:x21=0x0
h1:x22=0x0
h1:x23=0x0
h1:x24=0x0
h1:x25=0x0
h1:x26=0x0
h1:x27=0x0
h1:x28=0x0
h1:x29=0x0
h1:x30=0x0
h1:x31=0x0
- testbench-2hart.sv:164: Verilog $finish
regfile[10]= 0
regfile[10]= 0

```

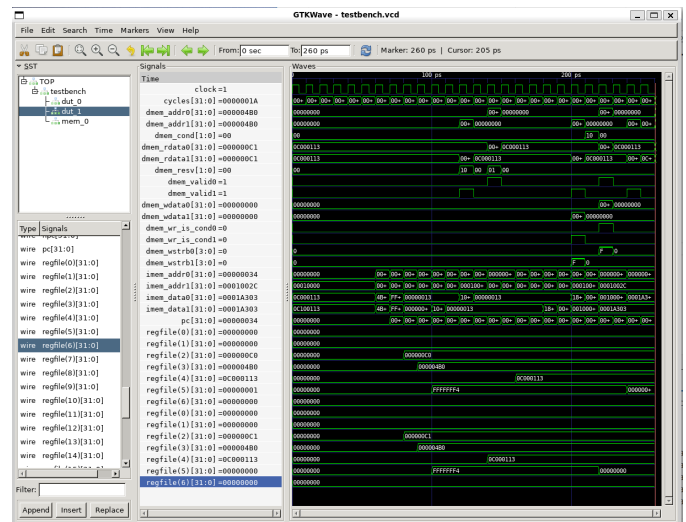
Label: test5-race-hart2-slowslow logfile

```

Built with Verilator 4.038 2020-07-11.
Recommended: Verilator 4.0 or later.
regfile[10]= 0
regfile[10]= 0
Simulated 20 cycles
h0:x0 =0x0
h0:x1 =0x80
h0:x2 =0xc0
h0:x3 =0x4b0
h0:x4 =0x8000093
h0:x5 =0x0
h0:x6 =0x8000093
h0:x7 =0x0
h0:x8 =0x0
h0:x9 =0x0
h0:x10=0x0
h0:x11=0x0
h0:x12=0x0
h0:x13=0x0
h0:x14=0x0
h0:x15=0x0
h0:x16=0x0
h0:x17=0x0
h0:x18=0x0
h0:x19=0x0
h0:x20=0x0
h0:x21=0x0
h0:x22=0x0
h0:x23=0x0
h0:x24=0x0
h0:x25=0x0
h0:x26=0x0
h0:x27=0x0
h0:x28=0x0
h0:x29=0x0
h0:x30=0x0
h0:x31=0x0
h1:x0 =0x0
h1:x1 =0x80
h1:x2 =0xc1
h1:x3 =0x4b0
h1:x4 =0x8000093

```

Label: test5-race-hart2-slowslow gtkwave



Label: test6-race-hart1-slow.s

```

/*
 * NERV -- Naive Educational RISC-V Processor
 *
 * Copyright (C) 2020 Claire Xenia Wolf
 * <claire@yosyshq.com>
 *
 * Permission to use, copy, modify, and/or
 * distribute this software for any
 * purpose with or without fee is hereby granted,
 * provided that the above
 * copyright notice and this permission notice
 * appear in all copies.
 *
 * THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR
 * DISCLAIMS ALL WARRANTIES

```

```

* WITH REGARD TO THIS SOFTWARE INCLUDING ALL
* IMPLIED WARRANTIES OF
* MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL
* THE AUTHOR BE LIABLE FOR
* ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL
* DAMAGES OR ANY DAMAGES
* WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR
* PROFITS, WHETHER IN AN
* ACTION OF CONTRACT, NEGLIGENCE OR OTHER
* TORTIOUS ACTION, ARISING OUT OF
* OR IN CONNECTION WITH THE USE OR PERFORMANCE OF
* THIS SOFTWARE.
*/

/**
 * @brief Two harts competing for the same
 * reservation. Hart0 attemptys the SC.W
 * one cycle later than Hart1 and so the H1
 * store is already in the pipeline.
 * The Reservation system must handle this,
 * and Hart0 should fail.
 * @author Rich Rademacher
 */

.section .text_H0,"ax",@progbits /* this H0
    section is "allocatable, program code,
    executable" */
.global main
.global _start
_start:
    addi x2, zero, 0xc0 /* new value to put in
    memory, 'core0' */
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
    should always change after SC.W */
    nop /* delay this hart by 1 cycle */
    lr.w x4, (x3)
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is actually in
    memory */
    ebreak

/* expected result:
    expect h0:x2 = 0xc0 (core 0)
    expect h0:x3 = 1200
    expect h0:x5 = 1 (fail)
    expect h0:x6 = 0xc1 (193)
*/

/**
 * @brief Hart1 requests memory and stores
 * immediately. Hart1 should succeed.
 */
.section .text_H1,"ax",@progbits /* this H1
    section is "allocatable, program code,
    executable" */
.global _start1
_start1:
    addi x2, zero, 0xc1 /* new value to put in
    memory, 'core1' */
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
    should always change after SC.W */
    lr.w x4, (x3)
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is actually in
    memory */
    ebreak

/* expected result:
    expect h1:x2 = 0xc1 (core 1 - succeed sc.w )
    expect h1:x5 = 0 (success)
    expect h1:x6 = 0xc1 (193)
*/

```

Label: test6-race-hart1-slow logfile

```

Built with Verilator 4.038 2020-07-11.
Recommended: Verilator 4.0 or later.
regfile[10]= 0
Simulated 18 cycles
h0:x0 =0x0
h0:x1 =0x0
h0:x2 =0xc0
h0:x3 =0x4b0
h0:x4 =0xc000113
h0:x5 =0x1
h0:x6 =0xc000113
h0:x7 =0x0
h0:x8 =0x0
h0:x9 =0x0
h0:x10=0x0
h0:x11=0x0
h0:x12=0x0
h0:x13=0x0
h0:x14=0x0
h0:x15=0x0
h0:x16=0x0
h0:x17=0x0
h0:x18=0x0
h0:x19=0x0
h0:x20=0x0
h0:x21=0x0
h0:x22=0x0
h0:x23=0x0
h0:x24=0x0
h0:x25=0x0
h0:x26=0x0
h0:x27=0x0
h0:x28=0x0
h0:x29=0x0
h0:x30=0x0
h0:x31=0x0
h1:x0 =0x0
h1:x1 =0x0
h1:x2 =0xc1
h1:x3 =0x4b0
h1:x4 =0xc000113
h1:x5 =0x0
h1:x6 =0xc000113
h1:x7 =0x0
h1:x8 =0x0
h1:x9 =0x0
h1:x10=0x0
h1:x11=0x0
h1:x12=0x0
h1:x13=0x0
h1:x14=0x0
h1:x15=0x0
h1:x16=0x0
h1:x17=0x0
h1:x18=0x0
h1:x19=0x0
h1:x20=0x0
h1:x21=0x0
h1:x22=0x0
h1:x23=0x0
h1:x24=0x0
h1:x25=0x0
h1:x26=0x0
h1:x27=0x0
h1:x28=0x0
h1:x29=0x0
h1:x30=0x0
h1:x31=0x0
- testbench-2hart.sv:164: Verilog $finish
regfile[10]= 0
regfile[10]= 0

```

[illegible]

```

/**
 * @brief Hart1 requests memory and stores
 *        immediately. Hart1 should succeed.
 */
.section .text_H1,"ax",@progbits /* this H1
    section is "allocatiable, program code,
    executable" */
.global _start1
_start1:
    addi x2, zero, 0xc1 /* new value to put in
        memory, 'core1' */
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
        should always change after SC.W */
    nop /* delay this hart by 1 cycle */
    lr.w x4, (x3)
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is actually in
        memory */
    ebreak

/* expected result:
    expect h1:x2 = 0xc1 (core 1)
    expect h1:x5 = 1 (fail)
    expect h1:x6 = 0xc0 (core 0 succeed)
*/

```

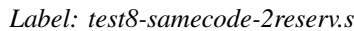
Label: test7-race-hart2-slow logfile

```

Built with Verilator 4.038 2020-07-11.
Recommended: Verilator 4.0 or later.
Simulated 18 cycles
h0:x0 =0x0
h0:x1 =0x0
h0:x2 =0xc0
h0:x3 =0x4b0
h0:x4 =0xc000113
h0:x5 =0x0
h0:x6 =0xc000113
h0:x7 =0x0
h0:x8 =0x0
h0:x9 =0x0
h0:x10=0x0
h0:x11=0x0
h0:x12=0x0
h0:x13=0x0
h0:x14=0x0
h0:x15=0x0
h0:x16=0x0
h0:x17=0x0
h0:x18=0x0
h0:x19=0x0
h0:x20=0x0
h0:x21=0x0
h0:x22=0x0
h0:x23=0x0
h0:x24=0x0
h0:x25=0x0
h0:x26=0x0
h0:x27=0x0
h0:x28=0x0
h0:x29=0x0
h0:x30=0x0
h0:x31=0x0
h1:x0 =0x0
h1:x1 =0x0
h1:x2 =0xc1
h1:x3 =0x12c0
h1:x4 =0xc000113
h1:x5 =0x0
h1:x6 =0xc000113
h1:x7 =0x0
h1:x8 =0x0
h1:x9 =0x0
h1:x10=0x0

```

Label: test7-race-hart2-slow gtwave



```

.section .text_H0,"ax",@progbits /* this H0
    section is "allocatiable, program code,
    executable" */
.global main
.global _start
_start:
    addi x2, zero, 0xc0 /* new value to put in
        memory, 'core0' */
    addi x3, zero, 1200 /* memory address 1200 */
    slli x3, x3, 0
    addi x5, zero, -12 /* dummy result...this
        should always change after SC.W */
    lr.w x4, (x3)
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is actually
        in memory */
    ebreak

/* expected result:
    expect h0:x2 = 0xc0 (core 0)
    expect h0:x3 = 1200
    expect h0:x5 = 0 (success)
    expect h0:x6 = 0xc0 (core 0)
*/

/**
 * @brief Hart1 code same as Hart0
 */
.section .text_H1,"ax",@progbits /* this H1
    section is "allocatiable, program code,
    executable" */
.global _start1
_start1:
    addi x2, zero, 0xc1 /* new value to put in
        memory, 'core1' */
    addi x3, zero, 1200 /* memory address 4800
        */
    slli x3, x3, 2
    addi x5, zero, -12 /* dummy result...this
        should always change after SC.W */
    lr.w x4, (x3)
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is
        actually in memory */
    ebreak

/* expected result:
    expect h1:x2 = 0xc1
    expect h1:x3 = 4800
    expect h1:x5 = 0 (success)
    expect h1:x6 = 0xc1 (core 1)
*/

```

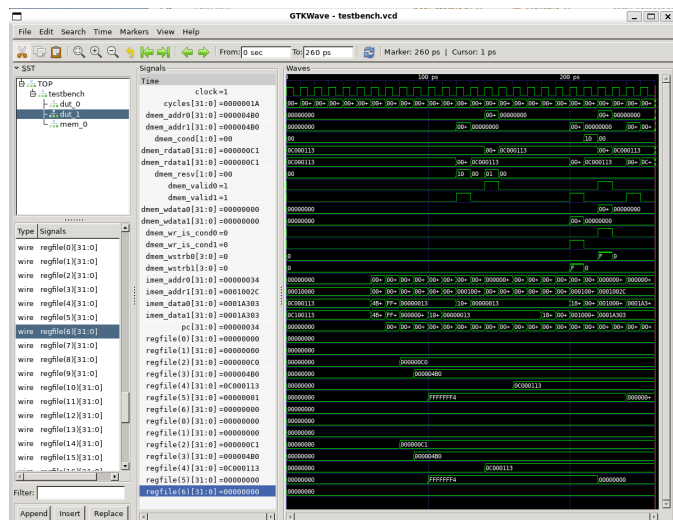
```
Built with Verilator 4.038 2020-07-11.
Recommended: Verilator 4.0 or later.
Simulated 17 cycles
h0:x0 =0x0
h0:x1 =0x0
h0:x2 =0xc0
h0:x3 =0xc8
h0:x4 =0xc000113
h0:x5 =0x0
h0:x6 =0xc000113
h0:x7 =0x0
h0:x8 =0x0
h0:x9 =0x0
h0:x10=0x0
h0:x11=0x0
h0:x12=0x0
h0:x13=0x0
h0:x14=0x0
h0:x15=0x0
h0:x16=0x0
```

```

h0:x17=0x0
h0:x18=0x0
h0:x19=0x0
h0:x20=0x0
h0:x21=0x0
h0:x22=0x0
h0:x23=0x0
h0:x24=0x0
h0:x25=0x0
h0:x26=0x0
h0:x27=0x0
h0:x28=0x0
h0:x29=0x0
h0:x30=0x0
h0:x31=0x0
h1:x0 =0x0
h1:x1 =0x0
h1:x2 =0xc1
h1:x3 =0x4b0
h1:x4 =0xc000113
h1:x5 =0x0
h1:x6 =0xc000113
h1:x7 =0x0
h1:x8 =0x0
h1:x9 =0x0
h1:x10=0x0
h1:x11=0x0
h1:x12=0x0
h1:x13=0x0
h1:x14=0x0
h1:x15=0x0
h1:x16=0x0
h1:x17=0x0
h1:x18=0x0
h1:x19=0x0
h1:x20=0x0
h1:x21=0x0
h1:x22=0x0
h1:x23=0x0
h1:x24=0x0
h1:x25=0x0
h1:x26=0x0
h1:x27=0x0
h1:x28=0x0
h1:x29=0x0
h1:x30=0x0
h1:x31=0x0
- testbench-2hart.sv:164: Verilog $finish
regfile[10]=          0
regfile[10]=          0

```

Label: test8-samecode-2reserv gtkwave



Label: test9-samecode-1reserv.s

```

/*
 * NERV -- Naive Educational RISC-V Processor
 *
 * Copyright (C) 2020 Claire Xenia Wolf
 * <claire@yosyshq.com>
 *
 * Permission to use, copy, modify, and/or
 * distribute this software for any
 * purpose with or without fee is hereby granted,
 * provided that the above
 * copyright notice and this permission notice
 * appear in all copies.
 */

/**
 * @brief test Two harts competing for the same
 * reservation at different pointers. Based on
 * the priority
 * you implement (H0=higher in this case) one
 * hart should fail
 * @author Rich Rademacher
 */

.section .text_H0,"ax",@progbits /* this H0
section is "allocatiable, program code,
executable" */
.global _start
_start:
    addi x2, zero, 0xc0 /* new value to put in
memory, 'core0' */
    addi x3, zero, 200 /* memory address 200 */
    addi x5, zero, -12 /* dummy result...this
should always change after SC.W */
    lr.w x4, (x3)
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is actually
in memory */
    ebreak

/* expected result:
expect h0:x2 = 0xc0 (core 0)
expect h0:x3 = 200
expect h0:x5 = 0 (success)
expect h0:x6 = 0xc0 (192)
*/

/**
 * @brief Hart1 code same as Hart0
 */
.section .text_H1,"ax",@progbits /* this H1
section is "allocatiable, program code,
executable" */
.global _start1
_start1:
    addi x2, zero, 0xc1 /* new value to put in
memory, 'core1' */
    addi x3, zero, 1200 /* memory address 1200
*/
    addi x5, zero, -12 /* dummy result...this
should always change after SC.W */
    lr.w x4, (x3)
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is
actually in memory */
    ebreak

/* expected result:
expect h1:x2 = 0xc1
expect h1:x3 = 1200
expect h1:x5 = 1 (fail)
expect h1:x6 = 0 (core 0 never wrote here)
*/

```

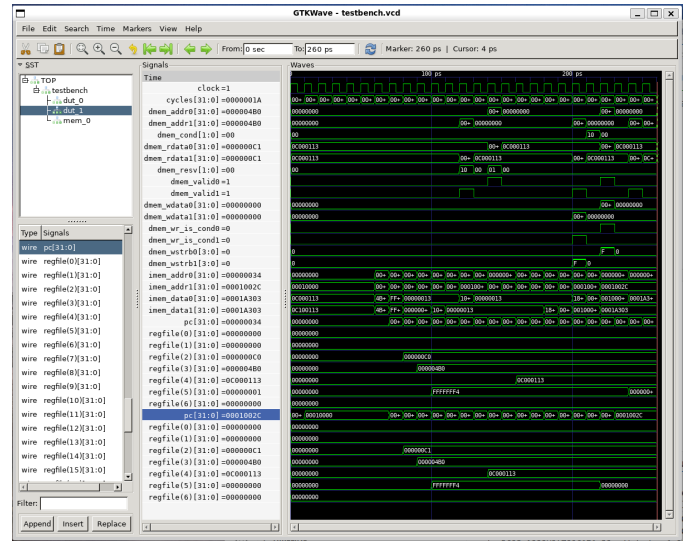
Label: test9-samecode-1reserv logfile

```

Built with Verilator 4.038 2020-07-11.
Recommended: Verilator 4.0 or later.
regfile[10]=      0
regfile[10]=      0
Simulated 21 cycles
h0:x0 =0x0
h0:x1 =0x0
h0:x2 =0xc0
h0:x3 =0x4b0
h0:x4 =0xc000113
h0:x5 =0x1
h0:x6 =0x0
h0:x7 =0x0
h0:x8 =0x0
h0:x9 =0x0
h0:x10=0x0
h0:x11=0x0
h0:x12=0x0
h0:x13=0x0
h0:x14=0x0
h0:x15=0x0
h0:x16=0x0
h0:x17=0x0
h0:x18=0x0
h0:x19=0x0
h0:x20=0x0
h0:x21=0x0
h0:x22=0x0
h0:x23=0x0
h0:x24=0x0
h0:x25=0x0
h0:x26=0x0
h0:x27=0x0
h0:x28=0x0
h0:x29=0x0
h0:x30=0x0
h0:x31=0x0
h1:x0 =0x0
h1:x1 =0x0
h1:x2 =0xc1
h1:x3 =0x4b0
h1:x4 =0xc000113
h1:x5 =0x0
h1:x6 =0xc000113
h1:x7 =0x0
h1:x8 =0x0
h1:x9 =0x0
h1:x10=0x0
h1:x11=0x0
h1:x12=0x0
h1:x13=0x0
h1:x14=0x0
h1:x15=0x0
h1:x16=0x0
h1:x17=0x0
h1:x18=0x0
h1:x19=0x0
h1:x20=0x0
h1:x21=0x0
h1:x22=0x0
h1:x23=0x0
h1:x24=0x0
h1:x25=0x0
h1:x26=0x0
h1:x27=0x0
h1:x28=0x0
h1:x29=0x0
h1:x30=0x0
h1:x31=0x0
- testbench-2hart.sv:164: Verilog $finish
regfile[10]=      0
regfile[10]=      0

```

Label: test9-samecode-1reserv gtkwave



Label: test10-hart1-inside-hart2.s

```

/*
 * NERV -- Naive Educational RISC-V Processor
 *
 * Copyright (C) 2020 Claire Xenia Wolf
 * <claire@yosyshq.com>
 *
 * Permission to use, copy, modify, and/or
 * distribute this software for any
 * purpose with or without fee is hereby granted,
 * provided that the above
 * copyright notice and this permission notice
 * appear in all copies.
 *
 */

/**
 * @brief Two harts competing for the same
 * reservation.
 * H0 gets a reservation before H1, but H1
 * writes back before H0
 * H1 should succeed since it committed first
 * @author Rich Rademacher
 */

.section .text_H0,"ax",@progbits /* this H0
section is "allocatable, program code,
executable" */
.global main
.global _start
_start:
    addi x2, zero, 0xc0 /* new value to put in
memory, 'core0' */
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
should always change after SC.W */
    lr.w x4, (x3)
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is actually in
memory */
    ebreak

/* expected result:
expect h0:x2 = 0xc0 (core 0)
expect h0:x3 = 1200

```

```

    expect h0:x5 = 1 (fail)
    expect h0:x6 = 0xc1 (core 1)
*/

/**
 * @brief hart1 Tries immediately to get the memory
 *        via LR/SC
 */
.section .text_H1,"ax",@progbits /* this H1
section is "allocatiable, program code,
executable" */
.global _start1
_start1:
    addi x2, zero, 0xc1 /* new value to put in
memory, 'core1' */
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
should always change after SC.W */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    lr.w x4, (x3)
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is actually in
memory */
    ebreak

/* expected result:
    expect h1:x2 = 0xc1 (core 1 )
    expect h1:x5 = 0 (success)
    expect h1:x6 = 0xc1 (core 1)
*/

```

```

h1:x4 =0xc000113
h1:x5 =0x1
h1:x6 =0xc000113
h1:x7 =0x0
h1:x8 =0x0
h1:x9 =0x0
h1:x10=0x0
h1:x11=0x0
h1:x12=0x0
h1:x13=0x0
h1:x14=0x0
h1:x15=0x0
h1:x16=0x0
h1:x17=0x0
h1:x18=0x0
h1:x19=0x0
h1:x20=0x0
h1:x21=0x0
h1:x22=0x0
h1:x23=0x0
h1:x24=0x0
h1:x25=0x0
h1:x26=0x0
h1:x27=0x0
h1:x28=0x0
h1:x29=0x0
h1:x30=0x0
h1:x31=0x0
- testbench-2hart.sv:164: Verilog $finish
regfile[10]= 0
regfile[10]= 0

```

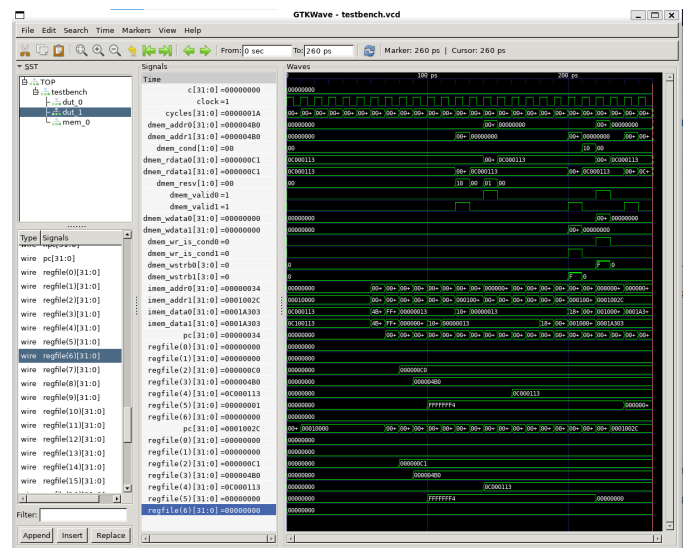
Label: test10-hart1-inside-hart2 logfile

```

Built with Verilator 4.038 2020-07-11.
Recommended: Verilator 4.0 or later.
regfile[10]= 0
regfile[10]= 0
Simulated 18 cycles
h0:x0 =0x0
h0:x1 =0x0
h0:x2 =0xc0
h0:x3 =0x4b0
h0:x4 =0xc000113
h0:x5 =0x0
h0:x6 =0xc000113
h0:x7 =0x0
h0:x8 =0x0
h0:x9 =0x0
h0:x10=0x0
h0:x11=0x0
h0:x12=0x0
h0:x13=0x0
h0:x14=0x0
h0:x15=0x0
h0:x16=0x0
h0:x17=0x0
h0:x18=0x0
h0:x19=0x0
h0:x20=0x0
h0:x21=0x0
h0:x22=0x0
h0:x23=0x0
h0:x24=0x0
h0:x25=0x0
h0:x26=0x0
h0:x27=0x0
h0:x28=0x0
h0:x29=0x0
h0:x30=0x0
h0:x31=0x0
h1:x0 =0x0
h1:x1 =0x0
h1:x2 =0xc1
h1:x3 =0x4b0

```

Label: test10-hart1-inside-hart2 gtkwave



Label: test11-hart1-interleave-hart2.s

```

/*
 * NERV -- Naive Educational RISC-V Processor
 *
 * Copyright (C) 2020 Claire Xenia Wolf
 * <claire@yosyshq.com>
 *
 * Permission to use, copy, modify, and/or
 * distribute this software for any
 * purpose with or without fee is hereby granted,
 * provided that the above
 * copyright notice and this permission notice
 * appear in all copies.
 */

```

```

* THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR
* DISCLAIMS ALL WARRANTIES
* WITH REGARD TO THIS SOFTWARE INCLUDING ALL
* IMPLIED WARRANTIES OF
* MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL
* THE AUTHOR BE LIABLE FOR
* ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL
* DAMAGES OR ANY DAMAGES
* WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR
* PROFITS, WHETHER IN AN
* ACTION OF CONTRACT, NEGLIGENCE OR OTHER
* TORTIOUS ACTION, ARISING OUT OF
* OR IN CONNECTION WITH THE USE OR PERFORMANCE OF
* THIS SOFTWARE.
*/

```

```
/**
```

```

* @brief Two harts competing for the same
* reservation.
* H1 gets a reservation before H0, but H0
* writes back before H1
* H0 should succeed since it committed first
* @author Rich Rademacher
*/

```

```

.section .text_H0,"ax",@progbits /* this H0
section is "allocatiable, program code,
executable" */
.global main
.global _start
_start:
    addi x2, zero, 0xc0 /* new value to put in
memory, 'core0' */
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
should always change after SC.W */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    lr.w x4, (x3)
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is actually in
memory */
    ebreak

```

```

/* expected result:
expect h0:x2 = 0xc0 (core 0)
expect h0:x3 = 1200
expect h0:x5 = 0 (success)
expect h0:x6 = 0xc0 (core 0)
*/

```

```
/**
```

```

* @brief hart1
*/
.section .text_H1,"ax",@progbits /* this H1
section is "allocatiable, program code,
executable" */
.global _start1
_start1:
    addi x2, zero, 0xc1 /* new value to put in
memory, 'core1' */
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
should always change after SC.W */
    lr.w x4, (x3)
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is actually in
memory */
    ebreak

```

```

/* expected result:
expect h1:x2 = 0xc1 (core 1)

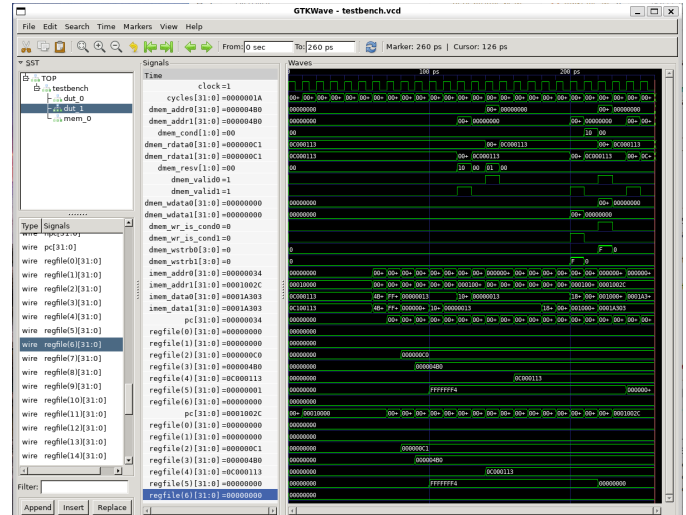
```

```

expect h1:x5 = 1 (fail)
expect h1:x6 = 0xc0 (core 0)
*/

```

Label: test11-hart1-interleave-hart2 gtkwave



Label: test12-hart1-interleave-hart2.s

```

/*
* NERV -- Naive Educational RISC-V Processor
*
* Copyright (C) 2020 Claire Xenia Wolf
* <claire@yosyshq.com>
*
* Permission to use, copy, modify, and/or
* distribute this software for any
* purpose with or without fee is hereby granted,
* provided that the above
* copyright notice and this permission notice
* appear in all copies.
*
*/

```

```
/**
```

```

* @brief Two harts competing for the same
* reservation.
* H0 and H1 are interleaved (H0 reserv, H1
* Resv, H0 write, H1 write)
* H0 should succeed since it committed first
* @author Rich Rademacher
*/

```

```

.section .text_H0,"ax",@progbits /* this H0
section is "allocatiable, program code,
executable" */
.global main
.global _start
_start:
    addi x2, zero, 0xc0 /* new value to put in
memory, 'core0' */
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
should always change after SC.W */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    lr.w x4, (x3)
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    sc.w x5, x2, (x3)

```

```

    lw x6, (x3)          /* read what is actually in
memory */
    ebreak

/* expected result:
expect h0:x2 = 0xc0 (core 0)
expect h0:x3 = 1200
expect h0:x5 = 0 (success)
expect h0:x6 = 0xc0 (core 0)
*/

/**
 * @brief hart1
 */
.section .text_H1,"ax",@progbits /* this H1
section is "allocatiable, program code,
executable" */
.global _start1
_start1:
    addi x2, zero, 0xc1 /* new value to put in
memory, 'core1' */
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
should always change after SC.W */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    lr.w x4, (x3)
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    sc.w x5, x2, (x3)
    lw x6, (x3)          /* read what is actually in
memory */
    ebreak

/* expected result:
expect h1:x2 = 0xc1 (core 1 )
expect h1:x5 = 1 (fail)
expect h1:x6 = 0xc0 (core 0)
*/

```

```

h0:x24=0x0
h0:x25=0x0
h0:x26=0x0
h0:x27=0x0
h0:x28=0x0
h0:x29=0x0
h0:x30=0x0
h0:x31=0x0
h1:x0 =0x0
h1:x1 =0x0
h1:x2 =0xc1
h1:x3 =0x4b0
h1:x4 =0xc000113
h1:x5 =0x1
h1:x6 =0x0
h1:x7 =0x0
h1:x8 =0x0
h1:x9 =0x0
h1:x10=0x0
h1:x11=0x0
h1:x12=0x0
h1:x13=0x0
h1:x14=0x0
h1:x15=0x0
h1:x16=0x0
h1:x17=0x0
h1:x18=0x0
h1:x19=0x0
h1:x20=0x0
h1:x21=0x0
h1:x22=0x0
h1:x23=0x0
h1:x24=0x0
h1:x25=0x0
h1:x26=0x0
h1:x27=0x0
h1:x28=0x0
h1:x29=0x0
h1:x30=0x0
h1:x31=0x0
- testbench-2hart.sv:164: Verilog $finish
regfile[10]= 0
regfile[10]= 0

```

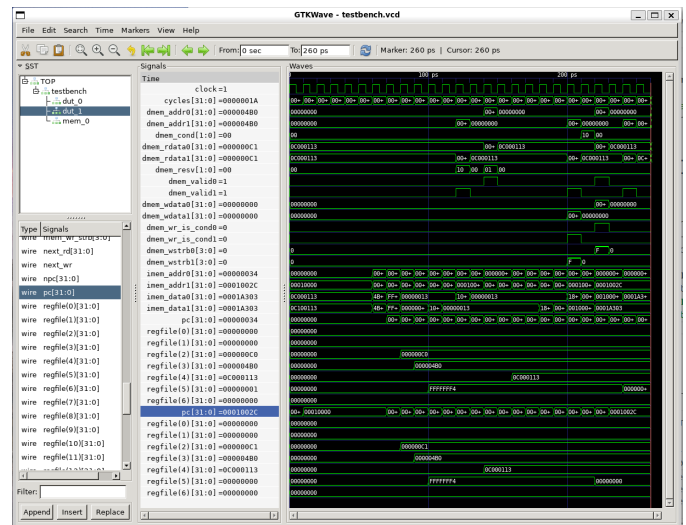
Label: test12-hart1-interleave-hart2 logfile

```

Built with Verilator 4.038 2020-07-11.
Recommended: Verilator 4.0 or later.
regfile[10]= 0
regfile[10]= 0
Simulated 25 cycles
h0:x0 =0x0
h0:x1 =0x0
h0:x2 =0xc0
h0:x3 =0x4b0
h0:x4 =0xc000113
h0:x5 =0x0
h0:x6 =0x0
h0:x7 =0x0
h0:x8 =0x0
h0:x9 =0x0
h0:x10=0x0
h0:x11=0x0
h0:x12=0x0
h0:x13=0x0
h0:x14=0x0
h0:x15=0x0
h0:x16=0x0
h0:x17=0x0
h0:x18=0x0
h0:x19=0x0
h0:x20=0x0
h0:x21=0x0
h0:x22=0x0
h0:x23=0x0

```

Label: test12-hart1-interleave-hart2 gtkwave



Label: test13-hart2-interleave-hart1.s

```

/*
 * NERV -- Naive Educational RISC-V Processor

```

```

*
* Copyright (C) 2020 Claire Xenia Wolf
* <claire@yosyshq.com>
*
* Permission to use, copy, modify, and/or
* distribute this software for any
* purpose with or without fee is hereby granted,
* provided that the above
* copyright notice and this permission notice
* appear in all copies.
*
*/

/**
 * @brief Two harts competing for the same
 * reservation.
 * H0 and H1 are interleaved (H1 reserv, H0
 * Resv, H1 write, H0 write)
 * H1 should succeed since it committed first
 * @author Rich Rademacher
 */

.section .text_H0,"ax",@progbits /* this H0
section is "allocatiable, program code,
executable" */
.global main
.global _start
_start:
    addi x2, zero, 0xc0 /* new value to put in
memory, 'core0' */
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
should always change after SC.W */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    lr.w x4, (x3)
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is actually in
memory */
    ebreak

/* expected result:
    expect h0:x2 = 0xc0 (core 0)
    expect h0:x3 = 1200
    expect h0:x5 = 1 (fail)
    expect h0:x6 = 0xc1 (core 1)
*/

/**
 * @brief hart1
 */
.section .text_H1,"ax",@progbits /* this H1
section is "allocatiable, program code,
executable" */
.global _start1
_start1:
    addi x2, zero, 0xc1 /* new value to put in
memory, 'core1' */
    addi x3, zero, 1200 /* memory address 0x1200 */
    addi x5, zero, -12 /* dummy result...this
should always change after SC.W */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    lr.w x4, (x3)
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    nop /* delay this hart by 1 cycle */
    sc.w x5, x2, (x3)
    lw x6, (x3) /* read what is actually in
memory */

```

```
ebreak
```

```

/* expected result:
    expect h1:x2 = 0xc1 (core 1 )
    expect h1:x5 = 0 (succeed)
    expect h1:x6 = 0xc1 (core 1)
*/

```

Label: test13-hart2-interleave-hart1 logfile

```

Built with Verilator 4.038 2020-07-11.
Recommended: Verilator 4.0 or later.
regfile[10]=          0
regfile[10]=          0
Simulated 25 cycles
h0:x0 =0x0
h0:x1 =0x0
h0:x2 =0xc0
h0:x3 =0x4b0
h0:x4 =0xc000113
h0:x5 =0x1
h0:x6 =0x0
h0:x7 =0x0
h0:x8 =0x0
h0:x9 =0x0
h0:x10=0x0
h0:x11=0x0
h0:x12=0x0
h0:x13=0x0
h0:x14=0x0
h0:x15=0x0
h0:x16=0x0
h0:x17=0x0
h0:x18=0x0
h0:x19=0x0
h0:x20=0x0
h0:x21=0x0
h0:x22=0x0
h0:x23=0x0
h0:x24=0x0
h0:x25=0x0
h0:x26=0x0
h0:x27=0x0
h0:x28=0x0
h0:x29=0x0
h0:x30=0x0
h0:x31=0x0
h1:x0 =0x0
h1:x1 =0x0
h1:x2 =0xc1
h1:x3 =0x4b0
h1:x4 =0xc000113
h1:x5 =0x0
h1:x6 =0x0
h1:x7 =0x0
h1:x8 =0x0
h1:x9 =0x0
h1:x10=0x0
h1:x11=0x0
h1:x12=0x0
h1:x13=0x0
h1:x14=0x0
h1:x15=0x0
h1:x16=0x0
h1:x17=0x0
h1:x18=0x0
h1:x19=0x0
h1:x20=0x0
h1:x21=0x0
h1:x22=0x0
h1:x23=0x0
h1:x24=0x0
h1:x25=0x0
h1:x26=0x0
h1:x27=0x0

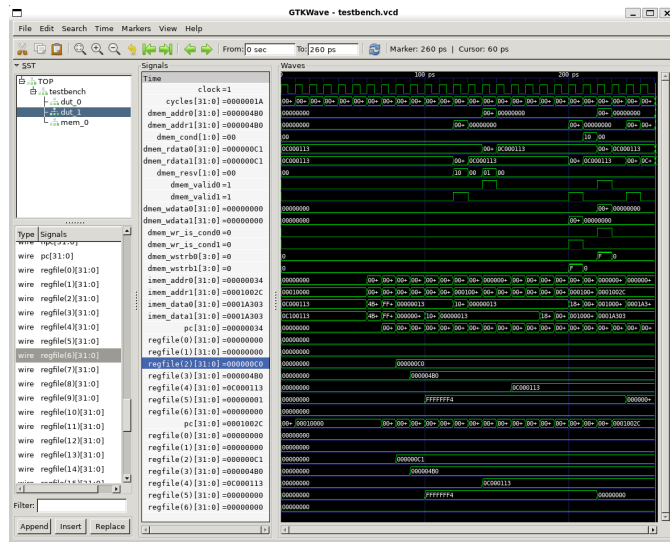
```

```

h1:x28=0x0
h1:x29=0x0
h1:x30=0x0
h1:x31=0x0
- testbench-2hart.sv:164: Verilog $finish
regfile[10]=      0
regfile[10]=      0

```

Label: test13-hart2-interleave-hart1 gtkwave



I hereby declare and acknowledge the usage of artificial intelligence as assistance in the preparation of this assignment. All responsibility for architectural choices, decision quality, and the final deliverable rests solely with me.

REFERENCES

- [1] eelectronics, "Single-Core vs Multi-Core Processor: A Detailed Comparison," *Medium*, Mar. 11, 2024. [Online]. Available: <https://eelectronics.medium.com/single-core-vs-multi-core-processor-a-detailed-comparison-8b29deac01c>
- [2] GeeksforGeeks, "What is a Multicore Processor?," *GeeksforGeeks*, Jun. 03, 2024. [Online]. Available: <https://www.geeksforgeeks.org/computer-organization-architecture/what-is-a-multicore-processor/>
- [3] Five Embedded, *RISC-V User ISA Manual*, Datasheet, 2025. [Online]. Available: <https://five-embeddev.com/riscv-user-isa-manual/Priv-v1.12/rv32.html#base-instruction-formats>. [Accessed: May 23, 2025].
- [4] Five EmbedDev, "RISC-V Instruction Set Manual, Volume I: RISC-V User-Level ISA," *Five EmbedDev*, 2021. [Online]. Available: <https://five-embeddev.com/riscv-user-isa-manual/Priv-v1.12/rv32.html#base-instruction-formats>. [Accessed: Oct. 21, 2025].
- [5] B. Goossens, "A Multicore RISC-V Processor with Multihart Cores," in *Undergraduate Topics in Computer Science*, pp. 401–423, Jan. 2023, doi: https://doi.org/10.1007/978-3-031-18023-1_13.